Week 1 – Industrial Project
Smart Student Processor

Student Profile Console Output:

```
Smart Student Profile Process

=== Student Readiness Capture System ===

Enter student full name: Josh
Enter age (16-100): 20
Enter grade percentage (0-100): 99
Days attended (0-30): 29
```

```
Smart Student Profile Process

=== PROCESSED STUDENT PROFILE ===

Name           : Josh
Age            : 20
Grade          : 99%
Attendance     : 29/30 days (97%)
---------------------------------------
Readiness (Full Assessment) :  READY
Readiness (Academic Only)   :  READY (Academic)

Press any key to exit...
```

Code:

```
using System;

namespace StudentProfileConsole
{
    class Program
    {

        const int TotalClassDays = 30;

        static void Main(string[] args)
        {
            Console.Title = "Smart Student Profile Processor (Console)";
            Console.WriteLine("=== Student Readiness Capture System ===\n");


            string name = GetValidatedString("Enter student full name: ");
            int age = GetValidatedInt("Enter age (16-100): ", 16, 100);
            int grade = GetValidatedInt("Enter grade percentage (0-100): ", 0, 100);
            int daysAttended = GetValidatedInt($"Days attended (0-{TotalClassDays}): ", 0, TotalClassDays);


            Student student = new Student(name, age, grade, daysAttended);


            string readiness1 = student.CalculateReadiness();
            string readiness2 = student.CalculateReadiness(grade);


            Console.Clear();
            Console.WriteLine("=== PROCESSED STUDENT PROFILE ===\n");
            student.DisplayProfile();
            Console.WriteLine($"Readiness (Full Assessment) : {readiness1}");
            Console.WriteLine($"Readiness (Academic Only)   : {readiness2}");
            Console.WriteLine("\nPress any key to exit...");
            Console.ReadKey();
        }
```

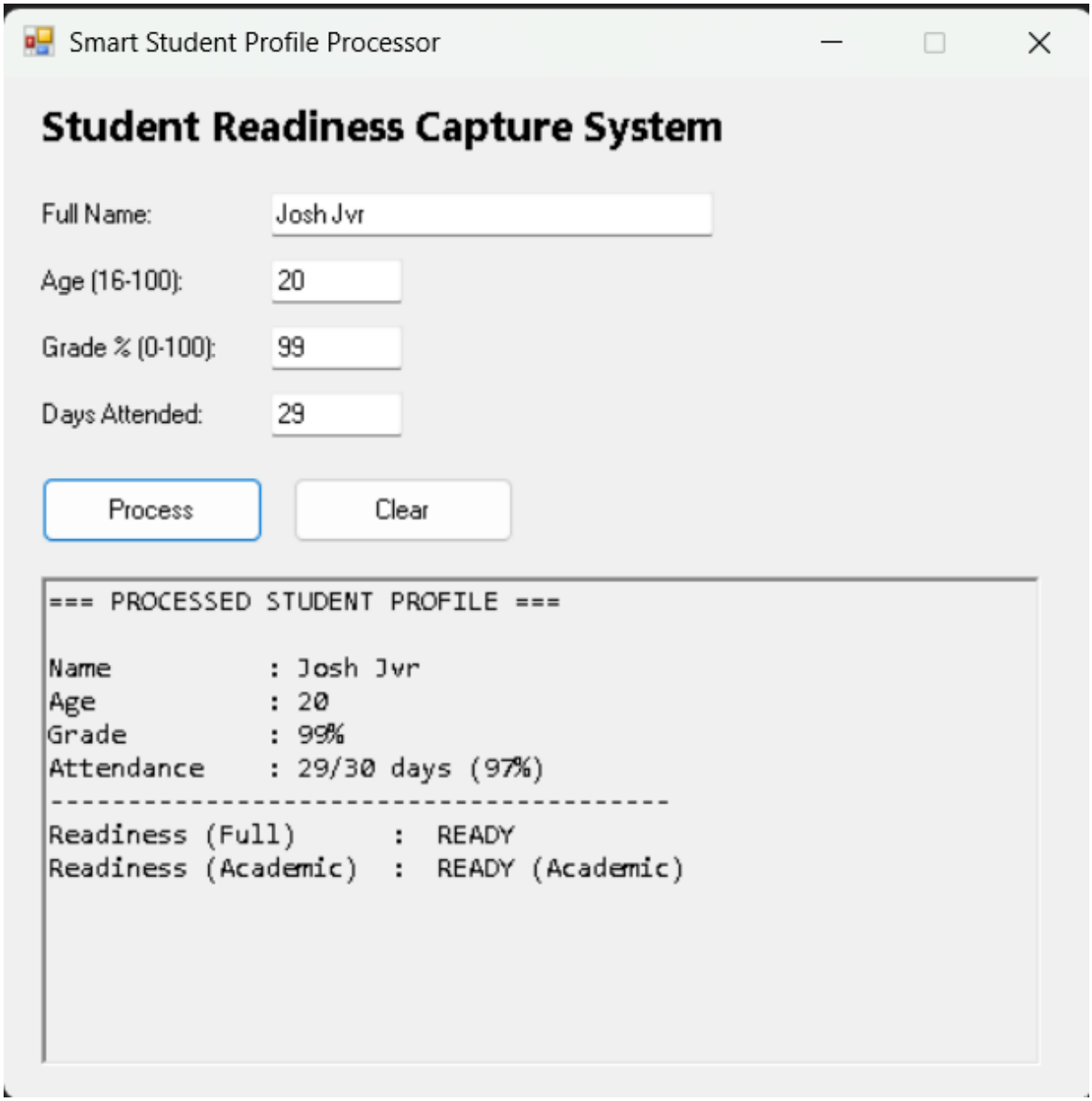
```
        static string GetValidatedString(string prompt)
        {
            string input;
            do
            {
                Console.Write(prompt);
                input = Console.ReadLine().Trim();
                if (string.IsNullOrWhiteSpace(input))
                    Console.WriteLine(" Name cannot be empty. Please try again.");
            } while (string.IsNullOrWhiteSpace(input));
            return input;
        }

        static int GetValidatedInt(string prompt, int min, int max)
        {
            int value;
            bool isValid;
            do
            {
                Console.Write(prompt);
                string input = Console.ReadLine();
                isValid = int.TryParse(input, out value) && value >= min && value <= max;
                if (!isValid)
                    Console.WriteLine($" Please enter a whole number between {min} and {max}.");
            } while (!isValid);
            return value;
        }
    }


    public class Student
    {

        private string name;
        private int age;
        private int gradePercent;
        private int daysAttended;
```

```
public class Student
{

    private string name;
    private int age;
    private int gradePercent;
    private int daysAttended;


    public string Name => name;
    public int Age => age;
    public int GradePercent => gradePercent;
    public int DaysAttended => daysAttended;


    private const int TotalDays = 30;


    public Student(string name, int age, int gradePercent, int daysAttended)
    {
        this.name = name;
        this.age = age;
        this.gradePercent = gradePercent;
        this.daysAttended = daysAttended;
    }


    public string CalculateReadiness()
    {

        double attendanceRatio = (double)daysAttended / TotalDays;
        int score = (int)(gradePercent * 0.6 + attendanceRatio * 100 * 0.4);

        if (score >= 70 && attendanceRatio >= 0.75)
            return " READY";
```

```
    public string CalculateReadiness()
    {

        double attendanceRatio = (double)daysAttended / TotalDays;
        int score = (int)(gradePercent * 0.6 + attendanceRatio * 100 * 0.4);

        if (score >= 70 && attendanceRatio >= 0.75)
            return " READY";
        else if (score >= 50 || attendanceRatio >= 0.5)
            return " CONDITIONAL";
        else
            return " NOT READY";
    }


    public string CalculateReadiness(int gradeOnly)
    {
        if (gradeOnly >= 70)
            return " READY (Academic)";
        else if (gradeOnly >= 50)
            return " CONDITIONAL (Academic)";
        else
            return " NOT READY (Academic)";
    }


    public void DisplayProfile()
    {
        Console.WriteLine($"Name          : {name}");
        Console.WriteLine($"Age           : {age}");
        Console.WriteLine($"Grade         : {gradePercent}%");
        Console.WriteLine($"Attendance    : {daysAttended}/{TotalDays} days ({(double)daysAttended / TotalDays:P0})");
        Console.WriteLine(new string('-', 40));
    }
}
}
```

Student Profile WInForm

Output:



Code:

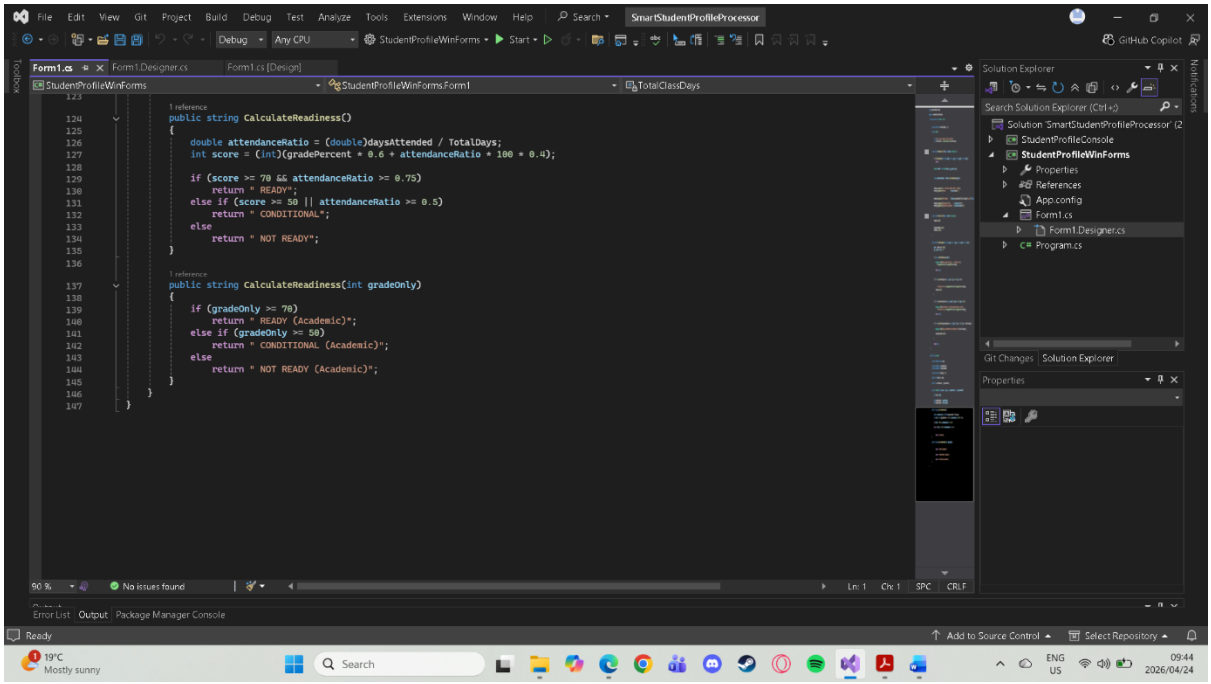
Form1.cs:

```
using System;
using System.Windows.Forms;

namespace StudentProfileWinForms
{
    3 references
    public partial class Form1 : Form
    {

        private const int TotalClassDays = 30;

        1 reference
        public Form1()
        {
            InitializeComponent();
            this.Text = "Smart Student Profile Processor";
            this.StartPosition = FormStartPosition.CenterScreen;
        }


        1 reference
        private void btnProcess_Click(object sender, EventArgs e)
        {

            if (!ValidateInputs(out string name, out int age, out int grade, out int days))
                return;


            Student student = new Student(name, age, grade, days);


            string readinessFull = student.CalculateReadiness();
            string readinessAcademic = student.CalculateReadiness(grade);


            rtbOutput.Clear();
            rtbOutput.AppendText("=== PROCESSED STUDENT PROFILE ===\n\n");
            rtbOutput.AppendText($"Name          : {student.Name}\n");
            rtbOutput.AppendText($"Age           : {student.Age}\n");
            rtbOutput.AppendText($"Grade         : {student.GradePercent}%\n");
            rtbOutput.AppendText($"Attendance    : {student.DaysAttended}/{TotalClassDays} days ({(double)student.DaysAttended / TotalClassDays:P0}
```

```
            rtbOutput.Clear();
            rtbOutput.AppendText("=== PROCESSED STUDENT PROFILE ===\n\n");
            rtbOutput.AppendText($"Name          : {student.Name}\n");
            rtbOutput.AppendText($"Age           : {student.Age}\n");
            rtbOutput.AppendText($"Grade         : {student.GradePercent}%\n");
            rtbOutput.AppendText($"Attendance    : {student.DaysAttended}/{TotalClassDays} days ({(double)student.DaysAttended / TotalClassDays:P0}
            rtbOutput.AppendText(new string('-', 40) + "\n");
            rtbOutput.AppendText($"Readiness (Full)      : {readinessFull}\n");
            rtbOutput.AppendText($"Readiness (Academic)  : {readinessAcademic}\n");
        }


        1 reference
        private void btnClear_Click(object sender, EventArgs e)
        {
            txtName.Clear();
            txtAge.Clear();
            txtGrade.Clear();
            txtDaysAttended.Clear();
            rtbOutput.Clear();
            txtName.Focus();
        }


        1 reference
        private bool ValidateInputs(out string name, out int age, out int grade, out int days)
        {
            name = txtName.Text.Trim();
            age = grade = days = 0;


            if (string.IsNullOrWhiteSpace(name))
            {
                MessageBox.Show("Name cannot be empty.", "Validation Error",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtName.Focus();
                return false;
            }
```

```
            if (!int.TryParse(txtAge.Text, out age) || age < 16 || age > 100)
            {
                MessageBox.Show("Age must be a whole number between 16 and 100.",
                    "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtAge.Focus();
                return false;
            }


            if (!int.TryParse(txtGrade.Text, out grade) || grade < 0 || grade > 100)
            {
                MessageBox.Show("Grade must be a whole number between 0 and 100.",
                    "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtGrade.Focus();
                return false;
            }


            if (!int.TryParse(txtDaysAttended.Text, out days) || days < 0 || days > TotalClassDays)
            {
                MessageBox.Show($"Days attended must be between 0 and {TotalClassDays}.",
                    "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                txtDaysAttended.Focus();
                return false;
            }

            return true;
        }
    }


    3 references
    public class Student
    {
        private readonly string name;
        private readonly int age;
        private readonly int gradePercent;
        private readonly int daysAttended;
```

```
    3 references
    public class Student
    {
        private readonly string name;
        private readonly int age;
        private readonly int gradePercent;
        private readonly int daysAttended;

        private const int TotalDays = 30;

        1 reference
        public string Name => name;
        1 reference
        public int Age => age;
        1 reference
        public int GradePercent => gradePercent;

        public int DaysAttended => daysAttended;

        1 reference
        public Student(string name, int age, int gradePercent, int daysAttended)
        {
            this.name = name;
            this.age = age;
            this.gradePercent = gradePercent;
            this.daysAttended = daysAttended;
        }


        public string CalculateReadiness()
        {
            double attendanceRatio = (double)daysAttended / TotalDays;
            int score = (int)(gradePercent * 0.6 + attendanceRatio * 100 * 0.4);

            if (score >= 70 && attendanceRatio >= 0.75)
                return " READY";
            else if (score >= 50 || attendanceRatio >= 0.5)
                return " CONDITIONAL";
            else
                return " NOT READY";
        }
```



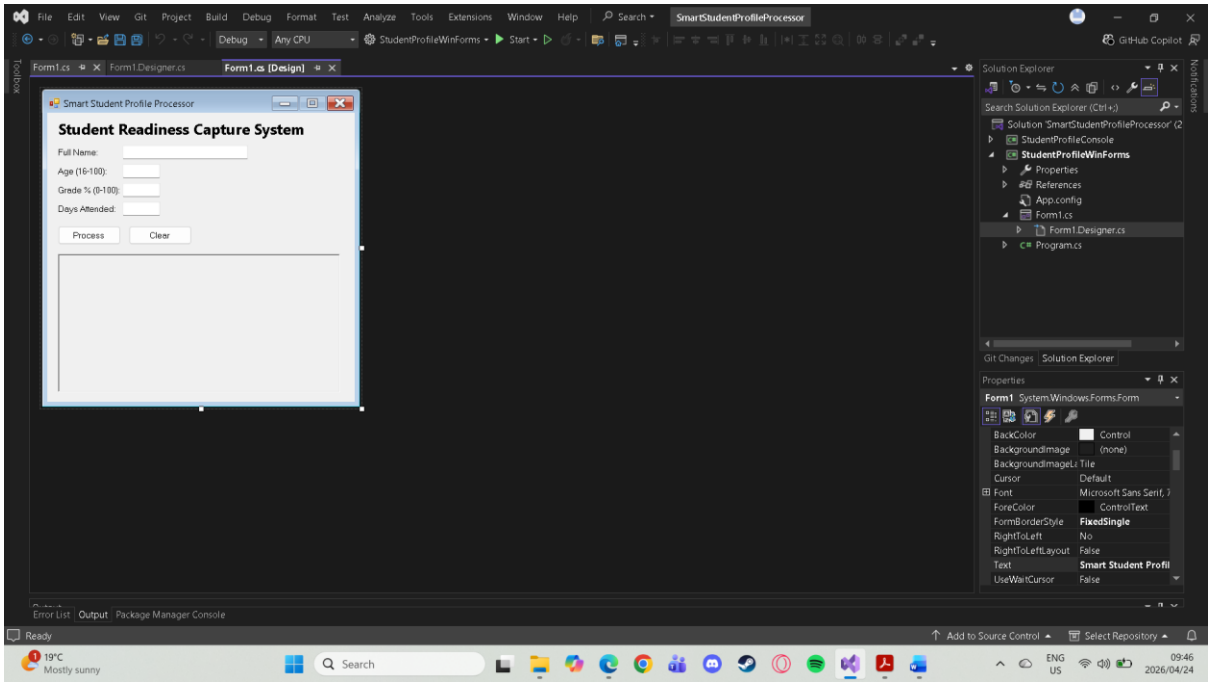
Form1 Designer.cs:

```
namespace StudentProfileWinForms
{
    3 references
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.Label lblTitle;
        private System.Windows.Forms.Label lblName;
        private System.Windows.Forms.TextBox txtName;
        private System.Windows.Forms.Label lblAge;
        private System.Windows.Forms.TextBox txtAge;
        private System.Windows.Forms.Label lblGrade;
        private System.Windows.Forms.TextBox txtGrade;
        private System.Windows.Forms.Label lblDaysAttended;
        private System.Windows.Forms.TextBox txtDaysAttended;
        private System.Windows.Forms.Button btnProcess;
        private System.Windows.Forms.Button btnClear;
        private System.Windows.Forms.RichTextBox rtbOutput;

        0 references
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
                components.Dispose();
            base.Dispose(disposing);
        }

        1 reference
        private void InitializeComponent()
        {
            this.lblTitle = new System.Windows.Forms.Label();
            this.lblName = new System.Windows.Forms.Label();
            this.txtName = new System.Windows.Forms.TextBox();
            this.lblAge = new System.Windows.Forms.Label();
            this.txtAge = new System.Windows.Forms.TextBox();
            this.lblGrade = new System.Windows.Forms.Label();
            this.txtGrade = new System.Windows.Forms.TextBox();
            this.lblDaysAttended = new System.Windows.Forms.Label();
            this.txtDaysAttended = new System.Windows.Forms.TextBox();
            this.btnProcess = new System.Windows.Forms.Button();
            this.btnClear = new System.Windows.Forms.Button();
```

```
        1 reference
        private void InitializeComponent()
        {
            this.lblTitle = new System.Windows.Forms.Label();
            this.lblName = new System.Windows.Forms.Label();
            this.txtName = new System.Windows.Forms.TextBox();
            this.lblAge = new System.Windows.Forms.Label();
            this.txtAge = new System.Windows.Forms.TextBox();
            this.lblGrade = new System.Windows.Forms.Label();
            this.txtGrade = new System.Windows.Forms.TextBox();
            this.lblDaysAttended = new System.Windows.Forms.Label();
            this.txtDaysAttended = new System.Windows.Forms.TextBox();
            this.btnProcess = new System.Windows.Forms.Button();
            this.btnClear = new System.Windows.Forms.Button();
            this.rtbOutput = new System.Windows.Forms.RichTextBox();
            this.SuspendLayout();

            // lblTitle
            this.lblTitle.AutoSize = true;
            this.lblTitle.Font = new System.Drawing.Font("Segoe UI", 14F, System.Drawing.FontStyle.Bold);
            this.lblTitle.Location = new System.Drawing.Point(12, 9);
            this.lblTitle.Name = "lblTitle";
            this.lblTitle.Size = new System.Drawing.Size(350, 25);
            this.lblTitle.Text = "Student Readiness Capture System";

            // lblName
            this.lblName.AutoSize = true;
            this.lblName.Location = new System.Drawing.Point(14, 55);
            this.lblName.Text = "Full Name:";

            // txtName
            this.txtName.Location = new System.Drawing.Point(120, 52);
            this.txtName.Size = new System.Drawing.Size(200, 23);

            // lblAge
            this.lblAge.AutoSize = true;
            this.lblAge.Location = new System.Drawing.Point(14, 85);
            this.lblAge.Text = "Age (16-100):";

            // txtAge
            this.txtAge.Location = new System.Drawing.Point(120, 82);
```

```
            // lblAge
            this.lblAge.AutoSize = true;
            this.lblAge.Location = new System.Drawing.Point(14, 85);
            this.lblAge.Text = "Age (16-100):";

            // txtAge
            this.txtAge.Location = new System.Drawing.Point(120, 82);
            this.txtAge.Size = new System.Drawing.Size(60, 23);

            // lblGrade
            this.lblGrade.AutoSize = true;
            this.lblGrade.Location = new System.Drawing.Point(14, 115);
            this.lblGrade.Text = "Grade % (0-100):";

            // txtGrade
            this.txtGrade.Location = new System.Drawing.Point(120, 112);
            this.txtGrade.Size = new System.Drawing.Size(60, 23);

            // lblDaysAttended
            this.lblDaysAttended.AutoSize = true;
            this.lblDaysAttended.Location = new System.Drawing.Point(14, 145);
            this.lblDaysAttended.Text = "Days Attended:";

            // txtDaysAttended
            this.txtDaysAttended.Location = new System.Drawing.Point(120, 142);
            this.txtDaysAttended.Size = new System.Drawing.Size(60, 23);

            // btnProcess
            this.btnProcess.Location = new System.Drawing.Point(17, 180);
            this.btnProcess.Size = new System.Drawing.Size(100, 30);
            this.btnProcess.Text = "Process";
            this.btnProcess.UseVisualStyleBackColor = true;
            this.btnProcess.Click += new System.EventHandler(this.btnProcess_Click);

            // btnClear
            this.btnClear.Location = new System.Drawing.Point(130, 180);
            this.btnClear.Size = new System.Drawing.Size(100, 30);
            this.btnClear.Text = "Clear";
            this.btnClear.UseVisualStyleBackColor = true;
            this.btnClear.Click += new System.EventHandler(this.btnClear_Click);
```

```
            // btnClear
            this.btnClear.Location = new System.Drawing.Point(130, 180);
            this.btnClear.Size = new System.Drawing.Size(100, 30);
            this.btnClear.Text = "Clear";
            this.btnClear.UseVisualStyleBackColor = true;
            this.btnClear.Click += new System.EventHandler(this.btnClear_Click);

            // rtbOutput
            this.rtbOutput.Font = new System.Drawing.Font("Consolas", 10F);
            this.rtbOutput.Location = new System.Drawing.Point(17, 225);
            this.rtbOutput.Size = new System.Drawing.Size(450, 220);
            this.rtbOutput.ReadOnly = true;

            // Form1
            this.ClientSize = new System.Drawing.Size(490, 460);
            this.Controls.Add(this.lblTitle);
            this.Controls.Add(this.lblName);
            this.Controls.Add(this.txtName);
            this.Controls.Add(this.lblAge);
            this.Controls.Add(this.txtAge);
            this.Controls.Add(this.lblGrade);
            this.Controls.Add(this.txtGrade);
            this.Controls.Add(this.lblDaysAttended);
            this.Controls.Add(this.txtDaysAttended);
            this.Controls.Add(this.btnProcess);
            this.Controls.Add(this.btnClear);
            this.Controls.Add(this.rtbOutput);
            this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
            this.Name = "Form1";
            this.Text = "Smart Student Profile Processor";
            this.ResumeLayout(false);
            this.PerformLayout();
        }
    }
}
```


File Edit View Git Project Build Debug Format Test Analyze Tools Extensions Window Help
Search
SmartStudentProfileProcessor
Debug
Any CPU
StudentProfileWinForms
Start
GitHub Copilot
Toolbox
Form1.cs
Form1.Designer.cs
Form1.cs [Design]
Smart Student Profile Processor
Student Readiness Capture System
Full Name:
Age (16-100):
Grade % (0-100):
Days Attended:
Process
Clear
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'SmartStudentProfileProcessor' (2
StudentProfileConsole
StudentProfileWinForms
Properties
References
App.config
Form1.cs
Form1.Designer.cs
Program.cs
Notifications
Git Changes
Solution Explorer
Properties
Form1 System.Windows.Forms.Form
BackColor Control
BackgroundImage (none)
BackgroundImageLa Tile
Cursor Default
Font Microsoft Sans Serif, 7
ForeColor ControlText
FormBorderStyle FixedSingle
RightToLeft No
RightToLeftLayout False
Text Smart Student Profil
UseWaitCursor False
Error List
Output
Package Manager Console
Ready
Add to Source Control
Select Repository
19°C
Mostly sunny
Search
ENG
US
09:46
2026/04/24